

Deadlocks

DSC 315: Computer Organization & Operating Systems

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



April 10 & 13, 2026

Table of Content

1 Deadlocks

- Introduction
- Handling Methods
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Deadlock Recovery

Recommended Reading From textbook:

Chapters 8.5, 8.6, 8.7, 8.8, 8.9

8.1

Introduction

Introduction to Deadlocks

What is a Deadlock?

- A **deadlock** is a situation where a set of processes are permanently blocked.
- Each process is waiting for a resource held by another process in the set.

Intuition:

- Process P_1 holds resource R_1 and waits for R_2
- Process P_2 holds resource R_2 and waits for R_1
- $\Rightarrow$ Neither can proceed

Why study deadlocks?

- Occur in operating systems and concurrent programs
- Can halt system progress completely
- Need strategies for **prevention, avoidance, and recovery**

System Model

Basic Assumptions

- System consists of a set of processes $\{P_1, P_2, \dots, P_n\}$
- A set of resource types $\{R_1, R_2, \dots, R_m\}$
- Each resource type may have multiple instances

Resource Usage Pattern

1 Request

- Process requests a resource
- If unavailable $\rightarrow$ process waits

2 Use

- Process uses the resource for execution

3 Release

- Process releases the resource after use

Note:

- Deadlock can occur if processes hold resources while waiting for others

Deadlock vs Livelock

Deadlock

- Processes are **blocked** and waiting indefinitely
- No process makes progress
- Caused by circular waiting for resources
- Example: P_1 waits for R_2 , P_2 waits for R_1

Livelock

- Processes are **active** but make no progress
- Continuously change state in response to each other
- Often due to overly polite resource releasing
- Example: two processes repeatedly retry and back off

Key Difference

- Deadlock: **no movement**
- Livelock: **movement without progress**

Necessary Conditions for Deadlock

A deadlock can occur only if all the following hold simultaneously:

1 Mutual Exclusion

- At least one resource is non-sharable

2 Hold and Wait

- Process holds resources while waiting for others

3 No Preemption

- Resources cannot be forcibly taken away

4 Circular Wait

- A cycle of processes exists, each waiting for the next

Key Idea:

- Breaking any one condition prevents deadlock

Handling Deadlocks

Strategies to deal with deadlocks:

1 Ignore the Problem

- Assume deadlocks are rare (“Ostrich approach”)
- Used in many general-purpose operating systems

2 Prevention or Avoidance

- **Prevention:** Break one of the necessary conditions
- **Avoidance:** Allocate resources carefully to stay in safe states

3 Detection and Recovery

- Allow deadlocks to occur
- Detect using algorithms (e.g., wait-for graph)
- Recover by terminating processes or preempting resources

Trade-off:

- Simplicity vs resource utilization vs overhead

8.5

Deadlock Prevention

Deadlock Prevention

Idea:

- Ensure that at least one necessary condition for deadlock never holds

Conditions to Break

- 1 Mutual Exclusion
- 2 Hold and Wait
- 3 No Preemption
- 4 Circular Wait

Key Point:

- Deadlock is **structurally prevented**, not dynamically handled

Deadlock Prevention Techniques

■ Mutual Exclusion

- Make resources sharable (if possible)

■ Hold and Wait

- Request all resources at once, or release before requesting new ones

■ No Preemption

- Preempt resources if a process cannot obtain additional ones

■ Circular Wait

- Impose a total ordering on resources
- Processes request resources in increasing order

Drawback:

- Breaking conditions is not possible mostly
- Low resource utilization, possible starvation

8.6

Deadlock Avoidance

Deadlock Avoidance

Idea:

- Allocate resources dynamically such that the system always remains in a **safe state**

Safe State

- A state where there exists a **safe sequence**
- All processes can complete in some order without deadlock

Key Requirement

- Each process must declare its **maximum resource requirement** in advance

Key Idea:

- Avoid unsafe states, not just deadlocks

Banker's Algorithm: Safe State Detection

Goal: Determine whether the system is in a safe state.

Data Structures:

- Available: Vector of available resources
- Max: Maximum demand of each process
- Allocation: Resources currently allocated
- Need = Max - Allocation

Algorithm:

- 1 Initialize:
 - Work = Available
 - Finish[i] = false for all i
- 2 Find a process i such that:
 - Finish[i] == false
 - Need[i] ≤ Work
- 3 If found:
 - Work = Work + Allocation[i]
 - Finish[i] = true
 - Add P_i to safe sequence
- 4 Repeat Step 2
- 5 If all Finish[i] == true → Safe state
Else → Unsafe state

Example: Given Data

Available = (3, 3, 2)

Process	Allocation	Max	Need
P0	(1,0,0)	(7,5,3)	(6,5,3)
P1	(2,0,0)	(3,2,2)	(1,2,2)
P2	(3,0,2)	(9,0,2)	(6,0,0)
P3	(2,1,1)	(2,2,2)	(0,1,1)
P4	(0,0,2)	(4,3,3)	(4,3,1)

Safe State Execution Steps

Initial Work = (3,3,2)

- P1: Need (1,2,2) $\leq$ Work $\Rightarrow$ Work = (5,3,2)
- P3: Need (0,1,1) $\leq$ Work $\Rightarrow$ Work = (7,4,3)
- P4: Need (4,3,1) $\leq$ Work $\Rightarrow$ Work = (7,4,5)
- P0: Need (6,5,3) $\leq$ Work $\Rightarrow$ Work = (8,4,5)
- P2: Need (6,0,0) $\leq$ Work $\Rightarrow$ Work = (11,4,7)

Safe Sequence:

$$P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$$

Conclusion:

- All processes can complete
- System is in a **SAFE STATE**

Example of Unsafe Sequence

Safe Sequence

Available = (3,3,2)

- $P1 \rightarrow \text{Work} = (5,3,2)$
- $P3 \rightarrow \text{Work} = (7,4,3)$
- $P4 \rightarrow \text{Work} = (7,4,5)$
- $P0 \rightarrow \text{Work} = (8,4,5)$
- $P2 \rightarrow \text{Work} = (11,4,7)$

Result:

- All processes complete
- System is SAFE

Unsafe Sequence Attempt

Available = (3,3,2)

- $P1 \rightarrow \text{Work} = (5,3,2)$
- Try $P0$:
 - Need = (6,5,3)
 - $(6,5,3) > (3,3,2) \Rightarrow$ Cannot proceed

Result:

- Sequence gets stuck
- Not a safe sequence

Unsafe State Example (No Safe Sequence)

Available = (0,0,0)

Process	Allocation	Max	Need
P0	(1,0,0)	(2,1,1)	(1,1,1)
P1	(1,1,0)	(2,2,1)	(1,1,1)
P2	(1,0,1)	(2,1,2)	(1,1,1)

Initial Work = (0,0,0)

Check all processes:

- P0: Need (1,1,1) $\not\leq$ Work $\Rightarrow$ Cannot proceed
- P1: Need (1,1,1) $\not\leq$ Work $\Rightarrow$ Cannot proceed
- P2: Need (1,1,1) $\not\leq$ Work $\Rightarrow$ Cannot proceed

Observation:

- No process satisfies $\text{Need} \leq \text{Work}$
- No process can execute

8.7

Deadlock Detection

Deadlock Detection

Idea:

- Allow deadlocks to occur, then detect them

When is it useful?

- When deadlocks are rare
- When prevention/avoidance is too costly

Approach:

- Periodically check system state
- Identify if a deadlock has occurred

Two Cases:

- Single instance of each resource type
- Multiple instances of resource types

Detection Algorithms

1. Single Instance (Wait-For Graph)

- Nodes = processes
- Edge $P_i \rightarrow P_j$ if P_i waits for P_j
- **Cycle** $\Rightarrow$ Deadlock

2. Multiple Instances

- Similar to Banker's safety algorithm
- Use:
 - Available, Allocation, Request
- Steps:
 - 1 Find process with Request $\leq$ Available
 - 2 Assume it finishes and releases resources
 - 3 Repeat
- If some processes cannot finish $\Rightarrow$ Deadlock

Key Point:

- Detect first, then recover

Deadlock Detection: Example

Available = (0,0,0)

Process	Allocation	Request
P0	(1,0,0)	(0,1,0)
P1	(0,1,0)	(0,0,1)
P2	(0,0,1)	(1,0,0)

Step 1: Work = (0,0,0), Finish = false for all

Step 2: Check processes

- P0: Request (0,1,0) $\not\leq$ Work
- P1: Request (0,0,1) $\not\leq$ Work
- P2: Request (1,0,0) $\not\leq$ Work

Result:

- No process can proceed
- All Finish[i] = false

Deadlock Exists

Banker's Algorithm vs Deadlock Detection

Deadlock Avoidance (Banker's Algorithm)

- Works **before allocation**
- Ensures system remains in a **safe state**
- Uses:
 - Max
 - Allocation
 - Need
- Decides whether to **grant or deny request**
- Prevents deadlock from occurring

Avoidance = Prevent

Deadlock Detection (with Multiple Instances)

- Works **after allocation**
- Allows deadlock to occur
- Uses:
 - Available
 - Allocation
 - Request
- Detects which processes are **deadlocked**
- Requires recovery mechanisms

Detection = Identify

8.8

Deadlock Recovery

After detecting a deadlock, how to recover?

1 Process Termination

- Abort all deadlocked processes, or
- Abort one process at a time until deadlock is resolved
- Selection based on priority, cost, resources held, etc.

2 Resource Preemption

- Temporarily take resources from processes
- Allocate to others to break deadlock
- Requires rollback to a safe state

Issues

- **Starvation:** same process repeatedly chosen
- **Overhead:** rollback and recomputation cost

Key Idea:

- Break the deadlock cycle by removing processes or resources

Textbook:

- 1 Avi Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Global Edition. Wiley, 2023. ISBN: 9781119320913.

Slides:

- 1 Avi Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts, Tenth Edition: Lecture Slides*. Wiley, 2018. Available at:
<https://os-book.com/OS10/slide-dir/index.html>. Accessed: 2026-03-06.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram

`laltu.sardar@iisertvm.ac.in`, `laltu.sardar.crypto@gmail.com`

Course webpage: https://laltu-sardar.github.io/courses/corgos_2026.html.